

AiBoO Security Platform

Real Deployment & Test Guide — run the full platform (no demo mode) on any system, and verify it end-to-end

Fact	Value
Repository	https://github.com/linux113/Aiboo
Guide applies to	branch main (after PR #1 merge) or the provided feature branch
Components run for real	MongoDB 7, Redis 7, Node backend (71 routes), Python agent API, web front door, React UI
What is NOT demo	no in-memory store, no seeded fake keys — real secrets, real DB persistence
One command	<code>scripts/real-stack.sh up</code> (Linux / macOS / Git-Bash on Windows)
Manual equivalent	Section 5 — any OS, incl. plain PowerShell
Test suite inside	10 end-to-end tests + 64-check smoke suite

Audience: engineers who receive this system and must run + demonstrate it on their own machine.

Contents

1. Architecture — what runs where
 2. Prerequisites per operating system
 3. Get the code
 4. One-command real run (`real-stack.sh`)
 5. Manual run, any OS (Windows PowerShell walkthrough)
 6. First login — admin bootstrap
 7. End-to-end tests (T1–T10) with exact commands
 8. Logs — every source, how to read them
 9. Debugging in VS Code (hybrid mode)
 10. Troubleshooting table
 11. Security notes for the real run
- Appendix A — Environment variables reference
- Appendix B — API quick reference
- Appendix C — Port map & file map
- Appendix D — Demo mode (only for presentations)

1. Architecture — what runs where

The REAL stack replaces every demo shim with the genuine component. MongoDB and Redis run as docker containers bound to loopback only; the backend, agent API and web front door run as real host processes you (or VS Code) start; the React UI is built once and served same-origin by the web front door, which proxies API and websocket traffic — so there is no CORS and it works behind any tunnel or domain.

Component	Technology	Port	Role
MongoDB 7	docker container aiboo-mongodb	27017 (127.0.0.1 only)	System of record — users, cameras, threats, incidents, audit. Persisted in docker volume mongo_data
Redis 7	docker container aiboo-redis	6379 (127.0.0.1 only)	Token blacklist + rate limiting shared store
Backend API	Node 20+ / Express, process	4000	71 REST routes + socket.io realtime; JWT auth, zod validation, pino logs
Agent API	Python / FastAPI, process	8001	Ingestion & zero-trust API for endpoint agents
Web front door	backend/web.mjs, Node process	5173	Serves frontend/dist; proxies /api + /socket.io -> :4000, /agent-api -> :8001 (mirrors production nginx)
React UI	frontend/dist (built once)	via :5173	Dashboard, surveillance grid, SOAR, agent console, audit, settings
CV service	Python, OPTIONAL (torch)	5050	Real camera inference — optional; tests simulate it via the ingest API

Request path when you open the app: browser -> http://localhost:5173 (web front door) -> static UI bundle; UI calls /api/* -> proxied to backend :4000; UI websocket /socket.io -> proxied with HTTP upgrade; agent console calls /agent-api/* -> proxied to :8001.

2. Prerequisites per operating system

Need	Windows 10/11	macOS	Linux
Docker	Docker Desktop (WSL2 backend), running	Docker Desktop or colima	docker engine + compose plugin
Node.js	20 LTS from nodejs.org	brew install node@20	nodesource or distro node 20+
Python	3.11+ from python.org (check "Add to PATH")	brew install python	python3 + pip
bash (for the one-command script)	Git Bash — installed with Git for Windows (includes openssl)	built-in	built-in
Git	git-scm.com	built-in after Xcode CLI tools	distro package

Version check (all OS):

```
docker --version # must print, docker daemon running
docker compose version # compose v2 plugin
node -v # v20.x or v22.x
python3 --version # 3.11+
openssl version # needed for secret generation
```

3. Get the code

```
git clone https://github.com/linux113/Aiboo.git
cd Aiboo/aiboo-platform
# stay on main, or checkout the branch you were given:
# git checkout <branch-name>
```

Everything below runs from **aiboo-platform/** (the folder containing docker-compose.yml, RUNBOOK.md, scripts/).

4. One-command real run — scripts/real-stack.sh

Works on Linux, macOS, and Windows via **Git Bash** (open "Git Bash" from the Start menu, cd into the folder). The script is idempotent — safe to re-run any time; it skips what is already up.

```
cd aiboo-platform
scripts/real-stack.sh up
```

What the first run does, step by step (be patient — ~5-10 min):

- **Secrets.** If backend/.env does not exist it is created from .env.example with four fresh openssl rand -hex 32 values: JWT_SECRET, AGENT_API_KEY, CV_INGEST_KEY, API_KEYS. File mode 600. The generated keys are printed once — save them.
- **Infrastructure.** docker compose --profile redis up -d mongodb redis — real MongoDB 7 + Redis 7 containers, ports bound to 127.0.0.1 only. The script WAITS for both healthchecks to report healthy (up to 90 s).
- **Dependencies.** npm ci for backend and frontend, pip install for the agent, only if missing.
- **UI build.** If frontend/dist is absent: vite build with same-origin env (VITE_API_URL=/api, VITE_SOCKET_URL=, VITE_AGENT_URL=/agent-api, VITE_CV_URL=/cv-api).
- **Real processes.** Starts three host processes with nohup, each writing a pid + a log file: backend (node server.js, with REDIS_URL and LOG_FILE), agent (python run_api.py), web front door (node web.mjs). Existing live processes are reused, not duplicated.
- **Health gate.** Polls http://localhost:4000/health until OK, then prints the URL table.

Expected final output:

```
[OK] mongodb healthy :27017 - redis healthy :6379
[OK] backend started (pid 12345) -> logs/backend.log
[OK] agent API started (pid 12346) -> logs/agent.log
[OK] web front door started (pid 12347) -> logs/web.log
[OK] backend healthy :4000
[OK] REAL STACK UP
UI http://localhost:5173 (first registered user = admin)
API http://localhost:4000 - agent :8001
logs tail -f logs/backend.log logs/agent.log logs/web.log
```

Daily commands:

```
scripts/real-stack.sh status # health of backend/agent/web + docker healthchecks + log listing
scripts/real-stack.sh logs # tail -f all three service logs (Ctrl-C to exit)
scripts/real-stack.sh down # stop processes + mongo/redis containers (DATA VOLUMES ARE KEPT)
```

5. Manual run, any OS — Windows PowerShell walkthrough

The exact equivalent of the script, command by command. Use this if you prefer separate terminals (recommended the first time — you see every log live) or if you do not want to use bash.

5.1 Secrets — copy the example and generate real keys (PowerShell):

```
cd aiboo-platform\backend
copy .env.example .env
# generate 4 secrets (repeat for JWT_SECRET, AGENT_API_KEY, CV_INGEST_KEY, API_KEYS):
-join ((1..64) | ForEach-Object { '{0:x}' -f (Get-Random -Max 16) })
# paste each value into .env replacing the <generate-with-openssl-rand-hex-32> placeholder
# (Git Bash / WSL / macOS/Linux users: openssl rand -hex 32)
```

5.2 Real infrastructure — MongoDB + Redis in docker (any OS):

```
cd aiboo-platform
docker compose --profile redis up -d mongodb redis
docker ps # wait until STATUS shows (healthy) for aiboo-mongodb and aiboo-redis
```

5.3 Backend — terminal 1 (PowerShell env syntax shown):

```
cd aiboo-platform\backend
npm ci
$env:REDIS_URL = "redis://localhost:6379"
$env:LOG_FILE = "$PWD\..\logs\backend.log" # optional: mirror logs to a file
mkdir ..\logs -Force | Out-Null
node server.js
# expect: "MongoDB connected" then "AiBoO Backend running on port 4000"
```

5.4 Agent API — terminal 2:

```
cd aiboo-platform\agent
python -m pip install -r requirements.txt
python run_api.py
# expect: Uvicorn running on http://0.0.0.0:8001
```

5.5 UI build + web front door — terminal 3 (build is once; later runs skip it):

```
cd aiboo-platform\frontend
npm ci
$env:VITE_API_URL="/api"; $env:VITE_SOCKET_URL=""; $env:VITE_AGENT_URL="/agent-api";
$env:VITE_CV_URL="/cv-api"
npx vite build
cd ..\backend
node web.mjs
# expect: "web front door on :5173 -> backend http://127.0.0.1:4000, agent
http://127.0.0.1:8001"
```

5.6 Open http://localhost:5173

Linux/macOS shorthand for each step: export VITE_API_URL=/api VITE_SOCKET_URL= VITE_AGENT_URL=/agent-api VITE_CV_URL=cv-api && npx vite build · REDIS_URL=redis://localhost:6379 LOG_FILE=../logs/backend.log node server.js · python3 run_api.py · node web.mjs

6. First login — admin bootstrap

- Open http://localhost:5173 — you land on the login screen.
- Click Register. Create your account (name, email, password 8+ chars).
- The FIRST registered user automatically becomes ADMIN — this is the documented bootstrap mechanism; the role field in the register form is ignored by design.
- Everyone registering after gets the ANALYST role.
- Data survives restarts (MongoDB volume). If you ever wipe the volume, the next first-user is admin again.

Token lifecycle: access JWT 1 h, refresh token in an httpOnly cookie 7 d — the UI rotates it silently. If you change JWT_SECRET in backend/.env, all existing logins become invalid (by design) — just log in again.

7. End-to-end tests — exact commands and expected results

Run these after the stack is up and you are logged in as admin. They walk the full detection -> correlation -> SOAR response chain against the REAL database. Keep the UI open in one window and run commands in the other — you will see the UI update live over the websocket.

PowerShell users: replace `curl -X POST -H ...` with `Invoke-RestMethod -Method Post -Uri <url> -Headers @{...} -ContentType 'application/json' -Body '...'` (examples for T4 included; the pattern is identical elsewhere).

T1 — Health of all three services

```
curl -s http://localhost:4000/health # {"status":"ok",...}
curl -s http://localhost:8001/health # {"status":"healthy","service":"AiBoO Ingestion..."}
curl -s -o NUL -w "%{http_code}" http://localhost:5173/ # 200 (Linux/macOS: -o /dev/null)
```

T2 — Authentication issues a real JWT

```
curl -s -X POST http://localhost:4000/api/auth/login -H "content-type: application/json" \
-d '{"email':'YOUR@MAIL','password':'YOURPASS'}'
# expect {"token":"eyJhbGciOiJIUzI1...", "refreshToken":...} — copy the token:
TOKEN=eyJ... # paste; used as "authorization: Bearer $TOKEN" below
```

Expected: a three-part JWT. Wrong password returns 401 Invalid credentials. Twenty auth calls in 15 minutes from one IP trip the rate limiter (429) — that is correct behaviour.

T3 — Add a camera (API + UI)

```
curl -s -X POST http://localhost:4000/api/cameras -H "authorization: Bearer $TOKEN" \
-H "content-type: application/json" \
-d '{"name':'Lobby','streamUrl':'rtsp://your-camera/stream','location':'Building A'}'
# expect 201 with the created camera JSON (note the _id — used in T4)
curl -s http://localhost:4000/api/cameras -H "authorization: Bearer $TOKEN"
# expect {"data":[...], "total":1, ...}
```

Required fields are **name** and **streamUrl**. In the UI the camera appears in the Surveillance grid immediately (socket event camera:added).

T4 — CV detection ingest (simulates the camera vision service) with the real ingest key

```
# CV_INGEST_KEY is printed by real-stack.sh on first run, or: grep CV_INGEST_KEY backend/.env
curl -s -X POST http://localhost:4000/api/cameras/detections \
-H "x-api-key: YOUR_CV_INGEST_KEY" -H "content-type: application/json" \
-d '{"cameraId':'CAMERA_ID_FROM_T3','type':'weapon_knife','severity':'critical','confidence':0.91}"
# expect 201 + detection JSON; a critical threat "weapon..." is auto-created and pushed live
```

PowerShell variant:

```
Invoke-RestMethod -Method Post -Uri http://localhost:4000/api/cameras/detections `
-Headers @{ "x-api-key" = "YOUR_CV_INGEST_KEY" } -ContentType "application/json" `
-Body '{ "cameraId": "CAMERA_ID", "type": "weapon_knife", "severity": "critical",
"confidence": 0.91 }'
```

Valid types include: person, vehicle, weapon, weapon_gun, weapon_knife, face_watchlist, fire, smoke, crowd, loitering, tamper, fall, tripwire (full list: backend/schemas/index.js). confidence accepts 0-1 (auto-normalised to %). Watch the UI: a critical notification pops and the threat lands in the Dashboard threat list within a second. A WRONG key returns 401 and is logged as a security warning — try it once to see the audit trail working.

T5 — Endpoint agent reporting (finding + heartbeat) with the real agent key

```
curl -s -X POST http://localhost:4000/api/agent/findings -H "x-api-key: YOUR_AGENT_API_KEY" \
-H "content-type: application/json" \
-d '{"agentId':'ws-finance-01','type':'c2_beacon','severity':'high','description':'beacon every 60s','host':'FIN-WS-12'}'
curl -s -X POST http://localhost:4000/api/agent/heartbeat -H "x-api-key: YOUR_AGENT_API_KEY" \
-H "content-type: application/json" \
-d '{"agentId':'ws-finance-01','status':'healthy','metrics':{'cpu':22,'mem':48}}"
# both expect 200/201; the finding appears live in the UI Agent Console
```

T6 — Correlated event triggers a SOAR incident (playbook match)

```
curl -s -X POST http://localhost:4000/api/agent/correlated -H "authorization: Bearer $TOKEN" \
-H "content-type: application/json" \
-d '{"event_type':'ransomware_prelude','severity':'critical','description':'mass file rename
+ shadow copy delete','entity':'FIN-WS-12'}'
# expect 200; playbook "ransomware-prelude-containment" matches and creates a PENDING
incident
curl -s "http://localhost:4000/api/soar/incidents?status=pending" -H "authorization: Bearer
$TOKEN"
# expect total >= 1; note the incident _id for T7
```

Why pending, not automatic: the shipped playbook has mode "approval" — human sign-off is required before any response action runs. (Playbooks: GET /api/soar/playbooks.)

T7 — Approve the incident, actions execute

```
curl -s -X POST http://localhost:4000/api/soar/incidents/INCIDENT_ID/approve \
-H "authorization: Bearer $TOKEN"
# expect {"status":"executed","actions":[{"type":"isolate"}, {"type":"lock_perimeter"}, ...]}
curl -s http://localhost:4000/api/dashboard/kpis -H "authorization: Bearer $TOKEN"
# totalActions grows by 2
curl -s http://localhost:4000/api/audit -H "authorization: Bearer $TOKEN"
# the audit trail contains soar.approve + the response actions, with your user id
```

The same approval is available as a button in the UI SOAR section. Re-approving returns 409 Incident already executed — idempotency working.

T8 — Persistence proof (this is what demo mode can never do)

```
scripts/real-stack.sh down
docker volume ls # mongo_data still listed
docker ps # containers gone
scripts/real-stack.sh up
curl -s http://localhost:4000/api/auth/login ... # SAME credentials still work
curl -s http://localhost:4000/api/soar/incidents -H "authorization: Bearer $TOKEN"
# the executed incident from T7 is still there
```

Expected: login succeeds with the same account and every camera / threat / incident created in T3-T7 is still present — data lives in MongoDB, not in process memory.

T9 — Automated smoke suite (64 checks)

```
cd aiboo-platform/backend
node smoke-test.mjs
# expect final line: === 64 passed, 0 failed ===
```

Covers: auth (register/login/refresh/rotate/logout/revocation), RBAC, camera + detection ingest, threats, agent endpoints, SOAR, validation errors, security headers, request-id propagation.

T10 — Negative / security tests (do these too — they prove the guards)

```
# wrong service key -> 401 + "Invalid API key attempt" warning in logs/backend.log
curl -s -X POST http://localhost:4000/api/agent/findings -H "x-api-key: bogus" ...
# analyst cannot approve (403) - register a second user, try T7 with its token
# tampered JWT -> 401
curl -s http://localhost:4000/api/soar/incidents -H "authorization: Bearer abc.def.ghi"
# schema validation -> 400 with field-level issues
curl -s -X POST http://localhost:4000/api/cameras -H "authorization: Bearer $TOKEN" \
-H "content-type: application/json" -d '{"name':'x'}' # streamUrl required
```

Test results checklist

#	Test	Pass criteria	Pass?
T1	Health trio	200/OK on 4000, 8001, 5173	
T2	Login + JWT	token returned; wrong pass 401	
T3	Camera create	201; visible in UI grid	
T4	CV ingest + critical fan-out	201; live threat + notification in UI	
T5	Agent finding + heartbeat	201/200; Agent Console updates	
T6	Correlated -> incident	pending incident created	
T7	Approve -> executed	isolate + lock_perimeter; audit entries	
T8	Restart persistence	all data intact after down/up	
T9	Smoke suite	64 passed, 0 failed	
T10	Negative tests	401/403/400 as specified	

8. Logs — every source and how to read them

Log	Where	Format
Backend (primary)	logs/backend.log when started via real-stack.sh or LOG_FILE set; otherwise stdout of the process	pino JSON lines, one per event
Agent API	logs/agent.log (script) or the agent terminal	uvicorn/plain text
Web front door	logs/web.log	startup + proxy errors
MongoDB	docker logs aiboo-mongodb	mongod log
Redis	docker logs aiboo-redis	redis log
Security audit trail	GET /api/audit (admin JWT) or UI audit view; stored in MongoDB	who did what, when, target, metadata
UI browser console	F12 in the browser	socket connect state, failed calls

Annotated backend log line (JSON):

```
{
  "level": 30,
  "time": 1788453506630,
  "pid": 9598,
  "reqId": "8978a8e2-...",
  "msg": "POST /api/agent/heartbeat"
}
| | | \___ human-readable event
| | \___ per-request correlation id (echoed in every HTTP response as X-Request-Id)
| \___ epoch ms
\___ 10=trace 20=debug 30=info 40=warn 50=error
```

Log level is controlled by LOG_LEVEL in backend/.env (trace|debug|info|warn|error). LOG_FILE can point anywhere; the file sink is ADDED to stdout, nothing is lost. To follow everything: scripts/real-stack.sh Logs. For long-running machines, add logrotate on logs/*.log (or ship the JSON lines to your stack).

9. Debugging the real stack in VS Code (hybrid mode)

- Open the aiboo-platform folder (it carries the tracked .vscode workspace).
- Start infra once: Terminal -> Run Task -> "aiboo: infra up (mongo+redis via Docker)".
- F5 -> compound "Rocket — Real hybrid stack": launches Backend (dev), Agent (API mode) and Web front door together; breakpoints in backend JS and agent Python both hit; LOG_FILE and REDIS_URL are preset.
- Great demo breakpoint: backend/services/soar.service.js inside onCorrelatedAlert — fire T6 and watch the playbook match + incident creation step by step.

- Frontend: run task "aiboo: frontend dev (vite)" and the "Frontend (Chrome debug)" launch config to debug TSX with sourcemaps.
- Full walkthrough with screenshots-level detail: VSCODE_GUIDE.md in the repo.

10. Troubleshooting

Symptom	Cause	Fix
backend exits immediately: "production boot check failed"	JWT_SECRET missing/default/short	set a real openssl rand -hex 32 value in backend/.env
"MongoServerSelectionError"	mongo container not up / not healthy yet	docker ps; docker logs aiboo-mongodb; wait for (healthy)
connect ECONNREFUSED 127.0.0.1:6379 (warn only)	Redis not running; backend falls back to in-memory stores	docker compose --profile redis up -d redis; single-instance runs are fine without it
Port already in use (4000/5173/27017)	another instance still running	scripts/real-stack.sh down; or kill the pid in logs/*.pid; lsof -i :4000
UI loads, login works, but lists stay empty	dist built without same-origin env (or stale old build)	rebuild: see 5.5 — set all four VITE_* vars exactly, then npx vite build; hard-refresh (Ctrl+Shift+R)
401 on every old session after redeploy	JWT_SECRET regenerated	log in again (tokens are signed with the secret)
CV ingest 401	wrong x-api-key	use CV_INGEST_KEY from backend/.env (printed by real-stack.sh first run)
Realtime updates not arriving	websocket blocked by a proxy in front of :5173	proxy must forward Upgrade for /socket.io (web.mjs already does; add the same to nginx)
real-stack.sh: "docker not found" on Windows	ran in PowerShell — it needs bash	use Git Bash, or follow Section 5 manual steps
npm ci fails on frontend	node version too old	node -v must be 20+

11. Security notes for the real run

- MongoDB and Redis are published on 127.0.0.1 ONLY — unreachable from other machines. Keep it that way; remote access belongs behind an authenticated reverse proxy.
- backend/.env holds live secrets: mode 600, never commit it (gitignored).
- All four service keys are compared timing-safe; wrong-key attempts are logged with source IP.
- First registered user = admin. On a shared machine, register immediately after first boot.
- Rate limiters: auth 20/15 min per IP, API 300/15 min, agent 600/15 min (tune in server.js).
- Every admin action (approvals, settings, user ops) lands in the immutable-style audit collection.
- For internet-facing demos: put the front door behind TLS (e.g. Caddy/nginx) and set TRUST_PROXY.

Appendix A — Environment variables that matter

Variable	Default	Meaning
MONGO_URI	mongodb://localhost:27017/aiboo	real DB connection
REDIS_URL	unset (in-memory fallback)	set to redis://localhost:6379 in the real run
JWT_SECRET	required, 32+ hex	signs all tokens — changing it invalidates sessions

Variable	Default	Meaning
JWT_ACCESS_TTL / JWT_REFRESH_TTL	1h / 7d	token lifetimes
AGENT_API_KEY	required	x-api-key for /api/agent/* ingest (endpoint agents)
CV_INGEST_KEY	required in prod	x-api-key for POST /api/cameras/detections (vision service)
API_KEYS	comma list	x-api-key accepted on protect routes for services
LOG_LEVEL / LOG_FILE	info / unset	verbosity; LOG_FILE mirrors all logs to a file
CORS_ORIGINS	dev list	only needed for split-origin dev (vite :5174)
SEED_DEMO_DATA	false	keep false for real runs
RATE_LIMIT_DISABLED	unset	debug only — never in production

Appendix B — API quick reference (used by the tests)

Method + Path	Auth	Purpose
GET /health	none	liveness
POST /api/auth/register login refresh logout	none / JWT	first user becomes admin
GET /api/auth/me	JWT	current user
GET/POST /api/cameras	JWT	name + streamUrl required
POST /api/cameras/detections	x-api-key CV_INGEST_KEY	vision ingest; critical types fan out
GET /api/cameras/detections	JWT	recent detections
GET/POST /api/threats	JWT	severity+title+source required; source enum
GET/POST /api/assets	JWT	inventory; drives systemsMonitored KPI
POST /api/agent/findings heartbeat correlated	x-api-key AGENT_API_KEY (correlated: JWT)	endpoint telemetry; correlated triggers SOAR
GET /api/soar/playbooks incidents	JWT	incident ?status=pending filter
POST /api/soar/incidents/:id/approve reject	JWT admin	executes response actions
GET /api/dashboard/kpis	JWT	activeThreats, systemsMonitored, actions...
GET /api/audit	JWT admin	security audit trail
GET /api/docs	JWT (prod)	Swagger UI; spec at /api/docs.json
GET /health, /events (agent, :8001)	agent key	agent service endpoints (proxied at /agent-api)

Appendix C — Port map & file map

Port	Service	Bind
4000	Backend API + socket.io	0.0.0.0 (host process)
5173	Web front door (UI + proxy)	0.0.0.0 (host process)
8001	Agent API (FastAPI)	0.0.0.0 (host process)

Port	Service	Bind
27017	MongoDB 7	127.0.0.1 ONLY (docker)
6379	Redis 7	127.0.0.1 ONLY (docker)
5050	CV service (optional)	0.0.0.0

File	What it is
scripts/real-stack.sh	one-command real stack (up/status/logs/down)
backend/web.mjs	same-origin front door + proxy (real & demo runs)
backend/.env	live secrets (generated; gitignored)
logs/backend.log, agent.log, web.log	service logs (LOG_FILE support)
RUNBOOK.md	full operations manual (docker/K8s/ops)
VSCODE_GUIDE.md	IDE debugging guide
backend/demo/*	in-memory DEMO mode — presentations only, see Appendix D

Appendix D — Demo mode (presentations only — NOT a real run)

For laptop demos with zero infrastructure there is an in-memory mode. It looks identical in the UI but holds data in process memory (wiped on restart) — use it only to show the screens:

```
cd aiboo-platform/backend
node demo/boot.mjs & # API :4000 on the in-memory store
node web.mjs & # front door :5173
node demo/seed-demo.mjs # admin@demo.io / Password123! + sample data
```

Everything else in this guide (tests, logs, UI behaviour) applies the same way. If data survives a restart you are on the real stack; if it resets, you are in demo mode — that is the quickest way to tell them apart.

Generated from the repository state of branch arena/01a0678a-aiboo (commits ec57036/c7514a8). Regenerate this document: `python3 scripts/make-run-guide-pdf.py`